Parsing Static Web Pages

Week 6 · Documents module · Textbook Chapter 6

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

September 21, 23 & 25, 2026

This week at a glance

We turn HTML — the language of web pages — into data you can analyze. Movies carry the examples all week: box office charts, Wikipedia film pages, and the Oscars.

Monday | Concepts & notebook intro

- Eight common shapes static pages hide their data in, and where each one sits on a single-site/multi-site, consistent/inconsistent design spectrum

Wednesday | Notebook lab

- Pilot each strategy in code: box office, Wikipedia, the Oscars, and Rotten Tomatoes

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 6

Companion notebook

`ch-06-static-pages.ipynb`

Bring a laptop

Wednesday is hands-on — we debug live.

1. Monday · Concepts

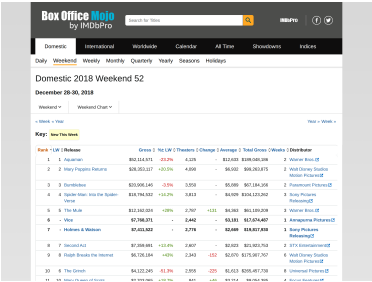
From HTML to data

The goal of scraping: convert one structured format — HTML — into another suited to analysis (a pandas DataFrame, CSV, or JSON).

HTML and XML are **siblings** (both descend from SGML), so your `BeautifulSoup` skills transfer directly.

But browsers *tolerate* malformed markup — unclosed tags, missing quotes, bad nesting — so real HTML is far messier than clean XML.

The core skill this week: **cutting through the noise** of navigation bars, ads, tracking scripts, and nested `<div>`s to reach the data.



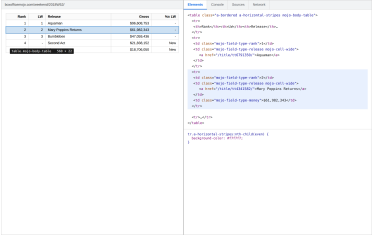
A page with the data already sitting in one table.

Inspect before you parse

Chapter 5 already put you in the Inspector: right-click the data, choose **Inspect**, and read the DOM the browser actually built.

Today that skill gets a narrower job. You are not exploring a page cold — you already know your target is data. The only question is **which shape the data is hiding in**, and every shape below answers that question with a different look inside the Inspector.

One habit from Chapter 5 carries forward unchanged: if an element is visible in the Inspector but missing from `requests.get(url).text`, JavaScript put it there after the fact — keep that in mind for Strategy 8.



The Elements panel on a real Box Office Mojo table.

Common research designs: a 2×2 spectrum

Before picking a parsing strategy, place your project on two axes: how many **sites** you're pulling from, and how **consistent** their design is page to page.

- **Single site, consistent design.** Fifty-two weekly Box Office Mojo pages — same template every time.
- **Single site, inconsistent design.** One oscars.org ceremony page where the winner's card and a nominee's card aren't shaped the same.
- **Multiple sites, consistent design.** English and French Wikipedia film infoboxes — same platform, same idea, different specifics.
- **Multiple sites, inconsistent design.** The Numbers, Box Office Mojo, and Rotten Tomatoes — three real sites, three unrelated HTML structures, for the same movies.

	1 site	**multi-site**
same design	weekly BOM charts	EN/FR infoboxes
differs	Oscars: winner vs. nominee	Numbers vs. Mojo vs. RT

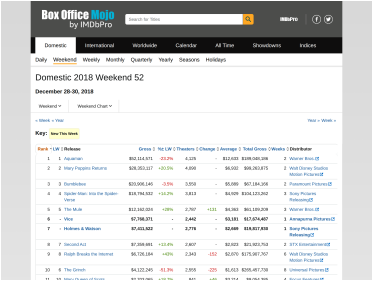
Complexity climbs left-to-right, top-to-bottom. Most real research projects live somewhere past the top-left cell.

Strategy 1 — the clean single table

The best case: one table, one target, nothing else on the page competing for it.

Box Office Mojo's weekly chart is exactly this — a single ranked table of that week's top movies. The moment you find it, you are basically done: no navigation tables, no decoys, no ambiguity about which element holds the data.

This is the case `pandas.read_html()` was built for.

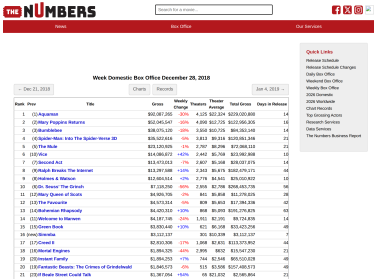


boxofficemojo.com/weekend/2018W52/

Strategy 2 — a table hiding among decoys

Real pages are rarely this polite. The Numbers' weekly chart has the *same kind* of data as Box Office Mojo, but its HTML ships **two** `<table>` elements — a full desktop layout and a separate mobile layout — and only one is the table you want.

Same site, same page, same intent as Strategy 1 — but now you must inspect and count before you trust `find_all("table")[0]`.



the-numbers.com — two tables, one is the data.

Strategy 3 — the table that needs cleanup

Sometimes the table extracts cleanly and the *values* don't. Wikipedia's "List of highest-grossing films" is the classic case: `read_html()` succeeds on the first try, and the real work starts after.

Footnote markers glue onto titles. A stray superscript letter sits in front of some dollar amounts. Numbers arrive as text. None of this breaks extraction — it breaks *analysis* until you clean it.

Rank	Peak	Title	Worldwide gross	Year	Ref
7	3	*Star Wars: The Force Awakens*	$2,068,223,624	2015	[# 12][# 13]
8	4	*Avengers: Infinity War*	$2,048,359,754	2018	[# 14][# 15]
9	6	*Spider-Man: No Way Home*	SM$1,922,598,800	2021	[# 16][# 17]
10	9	*Zootopia 2*	$1,866,647,950	2025	[# 18][# 19]
11	11	*The Odyssey* †	$1,721,314,740	2026	[# 20]
12	8	*Inside Out 2*	$1,698,863,816	2024	[# 21][# 22]
13	3	*Jurassic World*	$1,671,537,444	2015	[# 23][# 24]
14	7	*The Lion King*	$1,656,943,394	2019	[# 25][# 4]
15	3	*The Avengers*	$1,518,815,515	2012	[# 26][# 27]
16	4	*Furious 7*	$1,515,341,399	2015	[# 28][# 29]
17	11	*Top Gun: Maverick*	$1,503,260,455	2022	[# 30][# 31]
18	16	*Avatar: Fire and Ash*	$1,485,999,890	2025	[# 32][# 33]
19	10	*Frozen 2*	$1,450,026,933	2019	[# 34][# 35]
20	14	*Barbie*	$1,447,138,421	2023	[# 36][# 37]
21	5	*Avengers: Age of Ultron*	$1,402,809,540	2015	[# 38][# 29]

A table that parses fine and reads badly.

Strategy 4 — the infobox pattern

Not every page's real content lives inside a data `<table>`. Wikipedia's film infoboxes are a labeled key–value block — "Directed by," "Produced by," "Starring" — one row per fact, in roughly the same shape on every film's page.

Once you can pull one film's infobox apart, the same handful of lines pulls apart the next five hundred.

The Godfather

Theatrical release poster

Directed by Francis Ford Coppola

en.wikipedia.org — *The Godfather*.

Strategy 5 — porting the pattern to a new site

The infobox idea from Strategy 4 isn't just reusable across English Wikipedia pages — it travels to any site built on the same platform. French Wikipedia runs the same software and lays out its film infobox the same conceptual way.

But it is not the same page. The anchor that worked on the English article — a table class name — does not exist here at all; the reliable anchor turns out to be the table's `<caption>` instead. **The strategy transfers. The specific selector has to be re-verified, every time.**

Titre original	*The Godfather*
Réalisation	Francis Ford Coppola
Scénario	Mario Puzo Francis Ford Coppola
Musique	Nino Rota
Acteurs principaux	Marlon Brando Al Pacino James Caan Richard S. Castellano Robert Duvall
Sociétés de production	Paramount Pictures Alfran Productions
Pays de production	États-Unis
Genre	gangsters
Durée	175 minutes
Sortie	1972

fr.wikipedia.org — *Le Parrain*, same idea.

Strategy 6 — cards that need to become rows

The Academy Awards' own site lists nominees and winners as a grid of individual cards — one per category, holding several honorees — with **no table anywhere on the page**.

Turning this into a DataFrame means treating each honoree as one row: find the repeating category container, then pull category name, winner-or-nominee status, person, and film out of each honoree inside it.

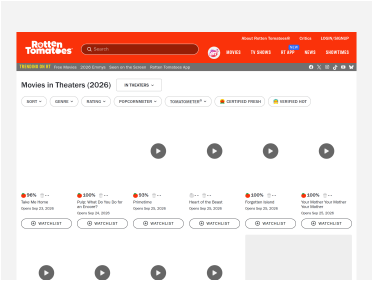
This is the shape behind most gallery and directory pages — and the one static-page pattern that shows up the most once you start looking for it.

oscars.org — 2026 ceremony, one card per honoree.

Strategy 7 — inconsistent cards, multiple sites

Rotten Tomatoes' "Movies in Theaters" grid looks like the same idea as the Oscars cards — a repeating tile per movie — but the tile itself is a different shape entirely: a poster, a title, and a Tomatometer score, built from custom elements instead of plain `<div>`s.

A research question that needs *both* box-office numbers and critical reception can't use one parser for both sites. You write one card-parser per site, then reconcile the two into a single schema — one row per movie — afterward. This is the general shape of most real scraping pipelines.



rottentomatoes.com — a differently-shaped card.

Strategy 8 — when “static” isn’t

Every strategy so far assumes the HTML you fetch already contains the data you want. IMDb’s own film pages break that assumption: a plain `requests.get()` against an IMDb title page returns HTTP **202** with **zero bytes** of body — not “missing the cast list,” completely empty.

The real content is built by JavaScript after a browser loads the page, and `requests` never runs JavaScript. A real browser sees the fully populated page on the right; a script using `requests` sees nothing at all.

What a browser renders. `requests` gets 0 bytes.

We come back to this once we’ve covered how to make a browser do the fetching for us.

Which strategy, and where on the spectrum?

In pairs, without writing any code: for each site below, name the strategy you'd reach for first, and place it on the 2×2.

1. A film festival's "official selections" page — posters and captions, no table
2. A newspaper's weekly bestseller list, archived and unchanging since 1995
3. IMDb's "Top 250" list page
4. A directory of every country's population, sourced from three different national statistics sites

Discussion

Which quadrant of the 2×2 is hardest — and is it hardest because of the *code*, or because of something else (rate limits, permission, trust in what a source reports)?

In the notebook

Wednesday's companion notebook pilots all eight strategies in code, one at a time:

- **Strategies 1–3** — box-office and Wikipedia tables: clean, decoy-hiding, and needs-cleanup
- **Strategies 4–5** — the Wikipedia infobox pattern, then porting it across a language edition
- **Strategies 6–7** — Oscars cards, then Rotten Tomatoes' differently-shaped cards
- **Strategy 8** — watching `requests` fail against IMDb, live

Connecting to Ch. 2 (Ethics)

Before pointing a loop at any site, check its `robots.txt` and Terms of Service, and reuse `responsible_get()` from the ethics chapter. Politeness is not optional.

2. Wednesday · Notebook Lab

Strategy 1 — the one-line win

One table, one call. When the page cooperates, take the win:

```
import pandas as pd

url = "https://www.boxofficemojo.com/weekend/2018W52/"
tables = pd.read_html(url)
print(f"{len(tables)} table(s) found") # 1 -- no ambiguity here

weekly = tables[0]
print(weekly.head())
```

No `requests`, no `BeautifulSoup` — `read_html()` fetches the page itself. Save this reflex for pages that look like Strategy 1; the next one won't be this easy.

Strategy 2 — finding the real table among decoys

Retrieve, then find the right table (pages hide data tables among navigation or mobile-layout tables):

```
import requests
from bs4 import BeautifulSoup

url = "https://www.the-numbers.com/box-office-chart/weekly/2018/12/28"
headers = {"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"}
soup = BeautifulSoup(requests.get(url, headers=headers).text, "html.parser")

tables = soup.find_all("table")
for i, table in enumerate(tables): # inspect each candidate
    print(i, len(table.find_all("tr")), "rows")
```

The one-line bug that shifts every column

Tempting shortcut:

```
# DON'T: silently drops empty cells
row.text.split("\n")
```

Correct — walk cell by cell so **empty cells stay aligned**:

```
cells = row.find_all(["td", "th"])
values = [c.get_text(strip=True)
          for c in cells]
```

A blank header cell (the “previous rank” column) is easy to lose.

Split-on-newline drops it. Every later value then shifts one column left, so **Movie** fills with distributor names.

The extra line of code buys correctness.

Strategy 3 — cleaning up after `read_html()`

```python
import pandas as pd

url = "https://en.wikipedia.org/wiki/List_of_highest-grossing_films"
df = pd.read_html(url, header=0)[0] # verified by inspection: index 0
print(df.dtypes) # "Worldwide gross" parsed as text
```

Expect to spend as long **cleaning** as extracting:

```python
# a stray footnote-reference letter can sit in front of the "$" --
# print a few raw values first so you know what you're stripping
df["gross"] = pd.to_numeric(
    df["Worldwide gross"].str.replace(r"[^0-9]", "", regex=True), errors="coerce")
df["footnote_count"] = df["Ref"].str.count(r"\[") # "[# 3][# 4]" -> 2
```

Strategy 4 — the infobox pattern

A table without a useful class name, but a very reliable shape — one `<th>`/`<td>` pair per row:

```
import requests
from bs4 import BeautifulSoup

url = "https://en.wikipedia.org/wiki/The_Godfather"
headers = {"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"}
soup = BeautifulSoup(requests.get(url, headers=headers).text, "html.parser")

infobox = soup.find("table", class_="infobox")
facts = {}
for row in infobox.find_all("tr"):
    label, value = row.find("th"), row.find("td")
    if label and value:
        facts[label.get_text(strip=True)] = value.get_text(" ", strip=True)
print(facts["Directed by"]) # 'Francis Ford Coppola'
```

Strategy 5 — porting the parser to a new site

The class name `infobox` doesn't exist on this page at all. Re-inspect, and anchor on the `<caption>` instead — same loop as Strategy 4 after that:

```
url = "https://fr.wikipedia.org/wiki/Le_Parrain_(film)"
soup = BeautifulSoup(requests.get(url, headers=headers).text, "html.parser")

infobox = soup.select("table:has(caption)")[0] # the anchor that transfers
facts = {}
for row in infobox.find_all("tr"):
    label, value = row.find("th"), row.find("td")
    if label and value:
        facts[label.get_text(strip=True)] = value.get_text(" ", strip=True)
print(facts["Titre original"]) # 'The Godfather' -- same film, new labels
```

The loop is identical. Only the anchor and the labels changed.

Strategy 6 — cards to rows: the Oscars parser

Navigate down through the real container hierarchy, category by honoree:

```
area = soup.find("div", class_="field--name-field-award-categories")
categories = area.find_all("div", class_="paragraph--type--award-category")

rows = list()
for cat in categories:
    name = cat.find(class_="field--name-field-award-category-oscars").get_text(strip=True)
    for h in cat.find_all("div", class_="paragraph--type--award-honoree"):
        rows.append({
            "category": name,
            "type": h.find(class_="field--name-field-honoree-type").get_text(strip=True),
            "entity": h.find(class_="field--name-field-award-entities").get_text(strip=True),
        })
df = pd.DataFrame(rows) # 24 categories on the 2026 ceremony page
```

Strategy 7 — a different card, a different site

Rotten Tomatoes' tiles are custom elements, not `<div>`s — same idea, new anchors:

```
url = "https://www.rottentomatoes.com/browse/movies_in_theaters/"
soup = BeautifulSoup(requests.get(url, headers=headers).text, "html.parser")

tiles = soup.find_all("tile-dynamic", attrs={"data-qa": "tile"})
records = list()
for tile in tiles:
    title = tile.find("span", class_="p--small")
    score = tile.find("rt-text", class_="critics-score")
    records.append({
        "title": title.get_text(strip=True) if title else None,
        "tomatometer": score.get_text(strip=True) if score else None,
    })
# join records on title with the Oscars/box-office DataFrames -- one row per movie
```

Strategy 8 — watching `requests` fail, live

```python
import requests

url = "https://www.imdb.com/title/tt0068646/"
headers = {"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"}
response = requests.get(url, headers=headers)
print(response.status_code, len(response.text))
# 202 0 -- zero bytes. Not a bad selector: there is nothing here to select.
```

This is not an error in your code

A 202 with an empty body means the server accepted the request and handed back a JavaScript shell instead of a page. `BeautifulSoup` has nothing to parse. This is the boundary this chapter's strategies don't cross — dynamic pages get their own week.

Make it reusable — and defensive

Abstract the working parser into a function; keep the `sleep` in the *caller*, not the function:

```
for year in range(2020, 2027):
    df = scrape_oscar_year(year, headers) # returns empty DataFrame on failure
    all_years.append(df)
    time.sleep(1) # politeness lives in the loop
```

Wrap fragile parsing so one bad page doesn't kill the run:

```
try:
    response.raise_for_status()
except requests.RequestException as e:
    print(f"Request failed: {e}"); return None
# AttributeError fires when .find() returns None and you call .text on it
# -- the single most common scraping error.
```

Error flavors

Some code here is *designed* to break (a guard demonstrating the `AttributeError` above) — that does not need a GitHub issue. Other code may be *genuinely* broken: confirm it, then search and file an issue.

Lab: work these, then show-and-tell

Work in pairs. Do these exercises from the notebook:

1. Get a Wikipedia table with `read_html`. Then make a visualization.
2. Write a parser for a page that has no table. Record your selectors.
3. Extend the parser to more than one page. Add a delay, error handling, and a function.
4. Parse a Wikipedia page that is not in English. Report the encoding problems that you correct.
5. Download each page one time into `cache/`. Then parse the files offline.
6. **5617**: examine the Terms of Service and the `robots.txt` file of two sites. Compare them with Fiesler et al. (2020)

Submitting

Start in class, finish at home, submit what you got working.

Graded on good-faith attempt to complete rather than getting everything working.

Show-and-Tell

Bring one of these to class: a bug you could not solve, data you find interesting, or a question for a research design.

Sites redesign without warning.
When a parser breaks, that's not a bug in your code —
it's the web working as designed.

Re-inspect, re-hypothesize, re-verify.

3. Friday · Textbook Revisions

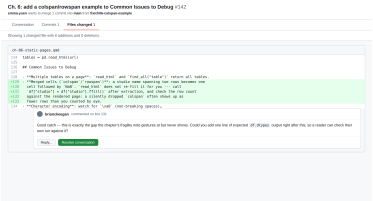
Improving Chapter 6 together

Today we make the book better. Each of you opens a **pull request** against `ch-06-static-pages.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable

Review comments still follow Week 3's claim / link / sharpen taxonomy — claim what you'll fix, link related issues, sharpen ones missing detail.



A revision menu for Chapter 6

High-value targets — pick one and scope it to a single PR:

- **Test the code against live sites.** The Numbers, Wikipedia, the Oscars, and Rotten Tomatoes all drift. Run the notebook, report what broke, and fix the selectors (this is the chapter's own "A Note on Fragility").
- **Close a gap in "Common Issues to Debug."** Add a worked `\xa0` / non-breaking-space example, or a `colspan/rowspan` case `read_html` mishandles.
- **Add a figure.** An annotated Inspector screenshot, or a diagram of the 2×2 research-design spectrum, would help readers place their own project.
- **Strengthen an exercise.** Add expected output, a rubric, or a starter cell so exercises are self-checking.
- **Extend "Further Reading."** A data-journalism scrape (ProPublica, The Markup) tied to the "Public Interest" callout.

A good revision, and a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **Archived web pages** — scraping the past with the Wayback Machine.

Key takeaways

1. Scraping is a pipeline: **retrieve** → **inspect** → **anchor** → **extract** → **DataFrame**.
2. Place your project on the single/multi-site × consistent/inconsistent spectrum before picking a strategy.
3. Tables, decoys, infoboxes, and cards are different shapes of the same problem — finding a reliable anchor.
4. Abstract into functions, rate-limit in the loop, and code defensively.
5. Pages change: the durable skill is **inspect, hypothesize, verify** — not memorized class names.